AI ENGINEERING NOTES · MODULE 02

Grounding AI with RAG & Context Engineering

A practitioner's study guide — how to get the *right* knowledge into a finite context window, anchored to the SEC Filing Analyst project.

Context Engineering | RAG Pipeline | Advanced RAG

Study guide · concepts in depth

Papers: Lewis 2020 · Karpukhin 2020 · Liu 2023

What's inside

> *Three concepts, in reading order. Each builds on the last: context engineering defines the problem, the RAG pipeline solves it, advanced RAG refines it.*

1 Context Engineering — the strategy

The prompt as a budget · the three failures of "paste everything" · lost in the middle · the four ingredients · placement · context rot

2 The RAG Pipeline — the mechanism

The index/query loop · chunking · embeddings & cosine similarity · vector databases & HNSW · retrieval & metadata filtering

3 Advanced RAG — the refinement

Re-ranking (bi- vs cross-encoder) · hybrid search (dense + BM25) · query rewriting · the full stack · why you must measure

How to read this. The code snippets are illustrative — they exist to make a mechanism concrete, not to be copy-pasted into production. The real, runnable project code comes when we build the copilot. Read for the mental models in the boxed takeaways.

PART 1 — THE STRATEGY

Context Engineering

> *What goes in the prompt, and why. This is the strategy layer; RAG is the mechanism that feeds it.*

THE ONE-SENTENCE TRUTH

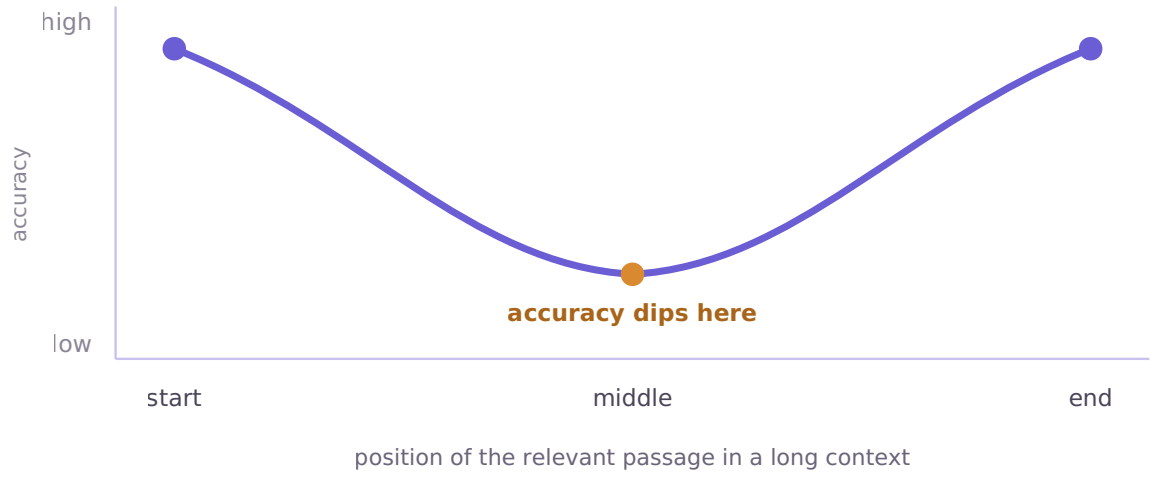
The prompt isn't one thing you write — it's a **budget** you allocate. Context engineering is the discipline of deciding, per request, what earns a seat in the finite context window, and *where* it sits.

Why this is a discipline, not just "writing a good prompt"

Module 1 left us with a hard limit: the context window is finite, and a single 10-K is ~80,000 tokens. The naive instinct — "models have 200k windows now, just paste the whole filing in" — fails for **three separate reasons**, and you need all three in your head.

- **Cost** — you pay per input token. Stuffing 80k tokens into *every* question is expensive at any real query volume.
- **Latency** — time-to-first-token scales with input length. Big contexts are slow contexts.
- **Accuracy** (the non-obvious one) — models get measurably *worse* at using information buried in the middle of a long context, even when it's right there. This is the **lost-in-the-middle** effect (Liu et al., 2023).

That third point surprises most engineers. It's not just "can it fit" — it's "can the model actually *use* what you gave it." Accuracy is highest when the relevant passage sits near the **start** or **end** of the context, and dips in the middle.

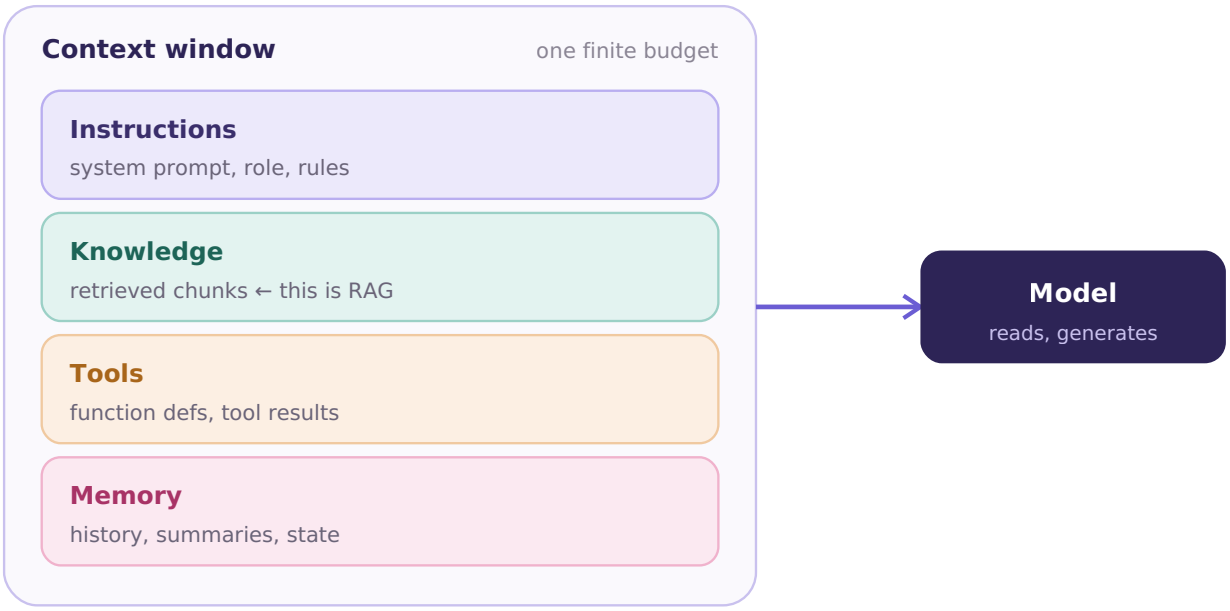


The lost-in-the-middle curve: the model reads the start and end best, the middle worst.

So even when 80,000 tokens *fit*, "paste everything" puts your risk-factors section in the model's blind spot. The module's goal reframes to: **don't give the model everything — give it the right thing, in the right place.**

The four ingredients competing for the budget

Everything in a context window is one of four things. They all draw from the same finite token budget, so they **compete**.



Four ingredients, one budget. More retrieved knowledge means less room for memory; more tools crowd out instructions.

The discipline is in the tradeoffs: there's no "just add more," only "what do I cut to make room." Mapped to the Filing Analyst:

- **Instructions** — cheap in tokens, high in leverage. The line *"answer only from the provided excerpts; if it's not there, say so"* is what later lets the copilot *refuse* instead of hallucinate.
- **Knowledge** — the retrieved 10-K chunks. Usually the biggest budget consumer. This is the slot RAG fills.
- **Tools** — function definitions cost tokens whether or not the model calls them.
- **Memory** — chat history grows over a session and quietly eats budget.

Placement: the lever most people don't pull

Because of lost-in-the-middle, *order* matters as much as content. Rules of thumb that hold up in practice:

- **Strongest retrieved chunk last** — closest to the question, in the high-accuracy zone.
- **Stable content at the top** — system instructions, tool definitions. Also helps prompt caching (you pay less for an unchanging prefix).

- **User's question at the very end**, after the evidence.

```
# a good default layout for a RAG prompt
[ system instructions ]      # top: stable, cacheable
[ tool definitions ]         # stable
[ retrieved chunk #3 (weakest) ]
[ retrieved chunk #2 ]
[ retrieved chunk #1 (strongest) ]   # strongest evidence nearest question
[ user question ]            # very end: high-accuracy zone
```

Context rot: the slow failure

Over a long agent run or chat, context fills with stale tool outputs, superseded answers, and dead ends. The model starts attending to irrelevant history — quality degrades even though nothing "broke." This is **context rot**, and the fixes are all curation: **summarize** old turns instead of carrying them verbatim, **evict** tool results once used, and **re-retrieve** fresh knowledge per question rather than accumulating it. You'll feel this hard in Module 3 (agents). For now: context is a resource you actively manage, not a log you append to.

Worked example: Filing Analyst

A user asks: *"What are Tesla's biggest stated risks this year?"*

The wrong way dumps the entire ~80,000-token 10-K then the question — failing on all three axes, with the risk-factors section buried mid-context where the model reads it worst. The context-engineered way:

```
[ system: "You answer only from the provided filing excerpts.
           Cite the section for every claim. If the answer
           isn't in the excerpts, say so." ]
[ excerpt: Risk Factors, supply chain     (chunk #3) ]
[ excerpt: Risk Factors, regulatory       (chunk #2) ]
[ excerpt: Risk Factors, competition      (chunk #1, strongest) ]
[ question: "What are Tesla's biggest stated risks this year?" ]
```

~2,000 tokens instead of 80,000, every piece in a high-attention position, and the instruction makes the model grounded and refusable. But notice the gap: **how did we know which excerpts to pull?** That selection problem is exactly what RAG solves — the next part.

MENTAL MODEL TO CARRY FORWARD

- The prompt is a **budget**, not a blank page.
- Four ingredients compete: **instructions, knowledge, tools, memory**.
- **Placement matters** (lost-in-the-middle) — strongest evidence and the question go near the end.
- Context is **curated, not accumulated** — watch for context rot.
- Context engineering decides *what* belongs; **RAG decides how to find it.**

PART 2 — THE MECHANISM

The RAG Pipeline

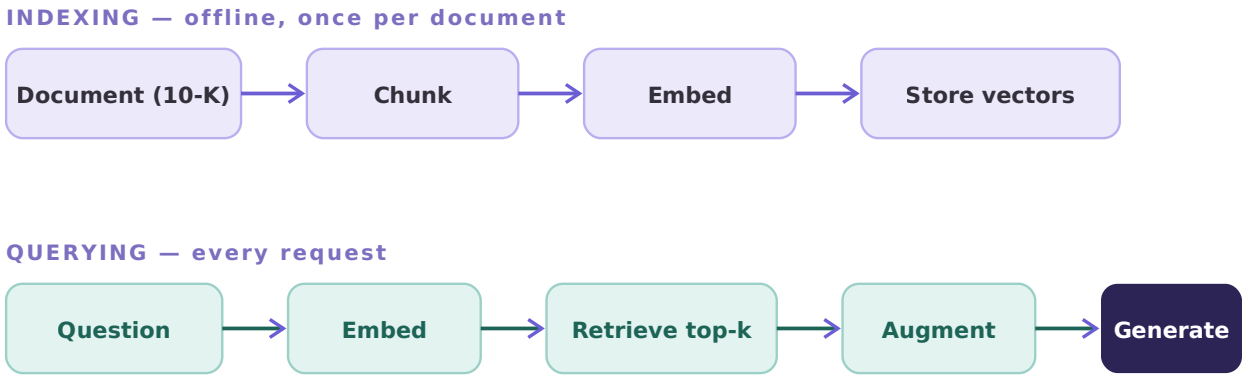
> *The mechanism that finds the right knowledge to fill the context window. Context engineering decided what belongs; RAG decides how to find it.*

THE ONE-SENTENCE TRUTH

Retrieval-Augmented Generation = chop documents into chunks, turn each into a vector that captures its *meaning*, store the vectors, and at query time fetch only the handful of chunks most relevant to the question — then hand those to the model.

The core loop

RAG has two phases. **Indexing** happens once, offline, when you ingest documents. **Querying** happens every time a user asks something.



Index once, query every time. The trick: question and chunks live in the same vector space, so matching becomes "find the nearest vectors."

Stage 1 — Chunking

You can't embed a whole 10-K as one vector — you'd lose all granularity. So you split it. **How** you split is the single most underrated decision in RAG; bad chunking quietly caps the quality of everything downstream.

Strategy	How it works	Verdict
Fixed-size	Split every N characters/tokens	Brutal — slices mid-sentence, mid-table
Recursive	Split on a hierarchy: paragraphs → sentences → words, going finer only when needed	The sane default
Semantic	Detect topic shifts (a similarity drop between adjacent sentences) and break there	Best quality, more compute

> **Why filings break naive chunkers.** 10-Ks aren't clean prose — they have section headers (Item 1A. Risk Factors), financial tables, footnotes, and boilerplate. A fixed-size splitter cuts a risk factor in half and splits a table across two chunks. For filings, use **structure-aware** chunking: split on the document's own section markers first, then recursively within sections.

Two dials: *chunk size* (too big dilutes the embedding into mush; too small loses context — ~500–1000 tokens is a common start for dense docs) and *overlap* (chunks share edge text so a boundary-straddling sentence survives whole — ~10–15%).

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=800,        # target size
    chunk_overlap=100,     # ~12% overlap so boundary sentences survive
    separators=["\n\n", "\n", ". ", " ", ""],  # paragraph, line, sentence...
)
chunks = splitter.split_text(filing_text)
```

Stage 2 — Embeddings

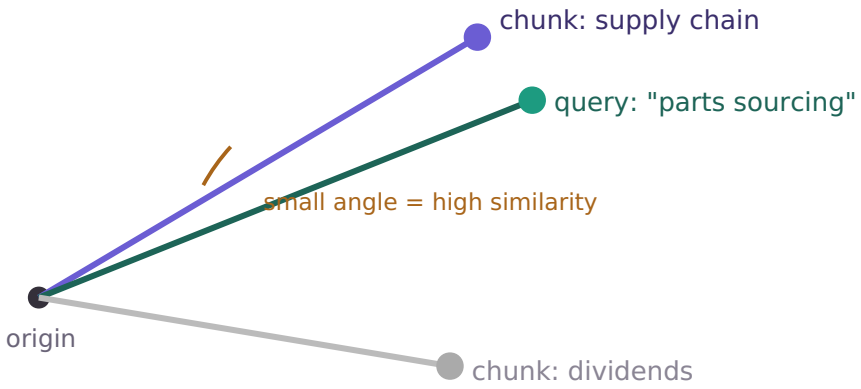
An **embedding** is a list of numbers (a vector) representing the *meaning* of text. Similar meaning lands at nearby points, regardless of exact words.

```
"supply chain disruption"   → [0.21, -0.08, 0.91, ...]  ┐ near
"problems sourcing parts"   → [0.19, -0.05, 0.88, ...]  ┘ each other
"quarterly dividend policy" → [-0.7,  0.42, 0.03, ...]    far away
```

This is *why* RAG can match a question to a chunk sharing **zero keywords** with it — and also RAG's weakness (it can miss exact strings), which is why we add keyword search back in Part 3.

How "nearby" is measured: cosine similarity

The standard metric is **cosine similarity** — the cosine of the angle between two vectors. It measures *direction* (semantic alignment), ignoring magnitude. Range: 1.0 = identical meaning, 0 = unrelated.

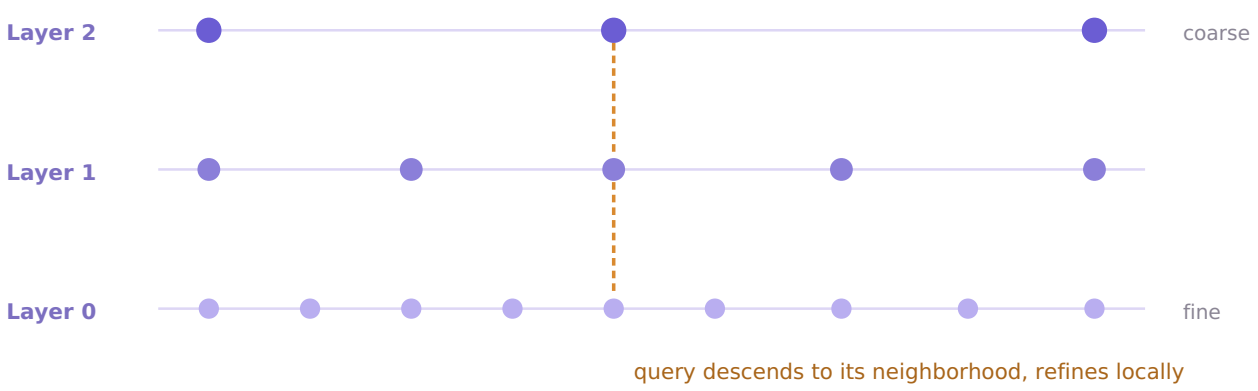


Vectors point somewhere in meaning-space; similarity is "how close do they point?" Small angle → high cosine similarity.

The rule you must respect. Embed your chunks and your queries with the **same model** — vectors from different models aren't comparable. Beyond that the tradeoffs are dimensions, max input length (must fit your chunk size), and API vs local. For learning and keeping the copilot cheap, a small local model like `all-MiniLM-L6-v2` is fine.

Stage 3 — Vector databases

With thousands of chunk-vectors, you need the nearest ones to a query vector *fast*. Brute-force comparison against every vector works at small scale but collapses at millions. Vector DBs use **Approximate Nearest Neighbor (ANN)** search — trading a sliver of accuracy for a massive speedup. The dominant algorithm is **HNSW** (Hierarchical Navigable Small World): a multi-layer graph where sparse top layers are "express lanes" for big jumps and dense lower layers do fine local search. A query enters at the top, hops toward its neighborhood, and descends.



HNSW: enter at sparse top layers for big jumps, descend to dense layers for fine search — log-ish time instead of a full scan.

Option	Shape	When
Chroma	In-process, dead simple	Learning & local dev — good first choice for the copilot
FAISS	Library (Meta), very fast	You manage storage yourself
pgvector	Vector search inside Postgres	You already run Postgres; vectors next to relational data
Pinecone / Weaviate / Qdrant / Milvus	Managed / production servers	When you outgrow local — filtering, scaling, hybrid built in

A vector DB stores three things per entry: the **vector**, the original **text** (to put in the prompt), and **metadata** (company, year, section, source URL — crucial for filtering and citations).

```
import chromadb
coll = chromadb.Client().create_collection("filings")
coll.add(
    documents=chunks,                       # text, for the prompt later
    embeddings=[v.tolist() for v in vectors],
    metadatas=[{"company": "TSLA", "year": 2025, "section": "Risk Factors"}] * len(chunks),
    ids=[f"tsla-2025-{i}" for i in range(len(chunks))],
)
```

Stage 4 — Retrieval

At query time: embed the question (same model!), ask the DB for the **top-k** nearest chunks, hand them to the model.

```
q_vec = model.encode(["What are Tesla's biggest stated risks this year?"])
results = coll.query(
    query_embeddings=[q_vec[0].tolist()],
    n_results=4,                                  # top-k
    where={"company": "TSLA", "year": 2025},      # metadata filter
)
```

Three retrieval levers: *top-k* (too few misses the answer; too many burns budget and reintroduces lost-in-the-middle — start k=3–5), *similarity metric* (cosine default), and *metadata filtering* — huge for filings. "Tesla's risks *this year*" should only search TSLA's 2025 filing; filtering by company and year *before* the vector search makes retrieval faster *and* far more accurate. This is the cheapest accuracy win you have.

Closing the loop: *augment* drops the chunks into the context-engineered prompt (strongest last); *generate* has the model answer, and because each chunk carries its `section` metadata, every claim can cite its source — the whole point of the Filing Analyst.

Where naive RAG falls short. Pure semantic search **misses exact terms** (a query for the literal phrase "substantial doubt" — the going-concern red flag — may not rank highest). The **top-k cut is blunt** (the best chunk might rank #7, just outside k=5). And **vague questions** retrieve vague chunks. Fixing these is Part 3.

MENTAL MODEL TO CARRY FORWARD

- RAG = **index once** (chunk → embed → store), **query every time** (embed → retrieve → augment → generate).
- **Chunking quality caps everything** — be structure-aware for filings.
- Embeddings put question and chunks in **one meaning-space**; cosine similarity finds neighbors.
- Vector DBs use **HNSW/ANN** for fast approximate nearest-neighbor search.
- **Metadata filtering** (company, year, section) is the cheapest accuracy win you have.

PART 3 — THE REFINEMENT

Advanced RAG

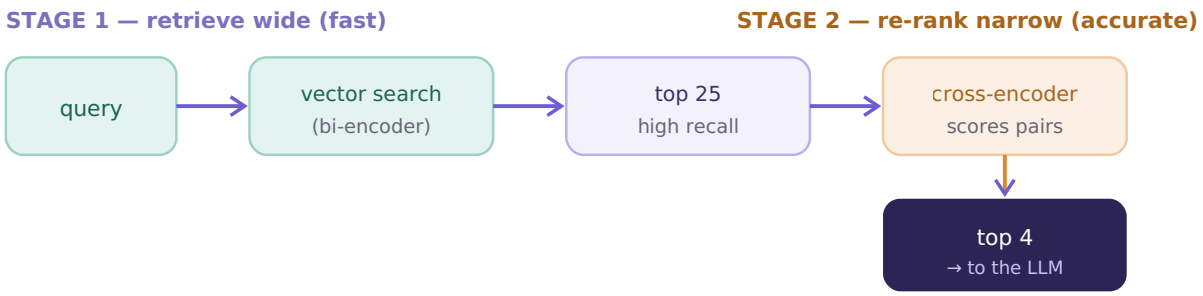
> *Naive RAG retrieves something relevant; these patterns make it retrieve the right thing in messy, real-world documents like filings.*

THE ONE-SENTENCE TRUTH

Naive top-k semantic search has three predictable failure modes — it misses exact terms, its cut is blunt, and it inherits messy queries. Re-ranking, hybrid search, and query rewriting each fix one.

Pattern 1 — Re-ranking (fixes the blunt cut)

The tension: you want high *recall* (don't miss the right chunk) *and* high *precision* (don't flood context with noise). Naive RAG forces a tradeoff via k. Re-ranking breaks it with a **two-stage retrieve**.



Cheap bi-encoder gets 25 candidates fast; expensive cross-encoder re-orders just those 25. The chunk that ranked #7 can jump to #2.

Bi-encoder vs cross-encoder is the key distinction. A *bi-encoder* (your embedding model) encodes query and chunk *separately* then compares vectors — fast, because chunks are pre-embedded, but it never sees them together. A *cross-encoder* takes `(query, chunk)` as a *single* input and outputs one relevance score — far more accurate, but it can't pre-compute, so it's too slow to run over the whole corpus. The combo is the insight: pay the slow model on 25 items, not 50,000.

```
from sentence_transformers import CrossEncoder
reranker = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
candidates = coll.query(query_embeddings=[q_vec], n_results=25)["documents"][0]
scored = reranker.predict([(question, c) for c in candidates])
top4 = [c for _, c in sorted(zip(scored, candidates), reverse=True)][:4]
```

> **If you add one advanced pattern, add this.** Re-ranking is usually the single highest-ROI upgrade to a naive RAG system.

Pattern 2 — Hybrid search (fixes the missed exact terms)

Semantic and keyword search fail in *opposite* ways, which is exactly why combining them works.

	Good at	Bad at
Dense (embeddings)	synonyms, paraphrase ("parts sourcing" ↔ "supply chain")	exact strings, codes, names, rare jargon
Sparse (BM25 keyword)	exact phrases ("substantial doubt", "Item 1A"), codes, names	synonyms and paraphrase

BM25 is the classic keyword-ranking algorithm — it scores a chunk by how often the query's terms appear, dampened by how common those terms are across the corpus (rare words count more). Decades old, fast, still excellent at literal matching. You run both retrievers and **fuse** their rankings, commonly with **Reciprocal Rank Fusion (RRF)**, which combines by *position* rather than raw scores — so you don't have to normalize a cosine score against a BM25 score (they live on different scales).

```
def reciprocal_rank_fusion(dense_ids, sparse_ids, k=60):
    scores = {}
    for rank, doc_id in enumerate(dense_ids):
        scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank)
    for rank, doc_id in enumerate(sparse_ids):
        scores[doc_id] = scores.get(doc_id, 0) + 1 / (k + rank)
    return sorted(scores, key=scores.get, reverse=True)
```

For the Filing Analyst this matters a lot: filing questions mix the semantic ("what are the risks") with the literal (dollar figures, exact legal phrases, section names). Many production DBs (Qdrant, Weaviate, pgvector + extensions) offer hybrid search natively.

Pattern 3 — Query rewriting (fixes the messy query)

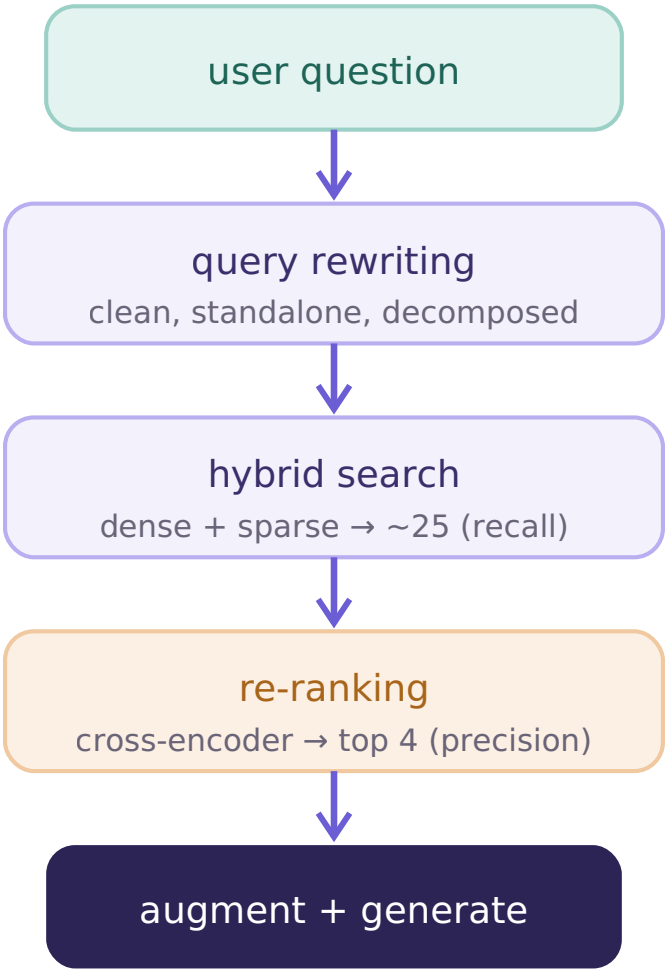
The retriever is only as good as the query it's given, and real questions are often bad retrieval queries — vague, conversational, multi-part, context-dependent. So transform the query *before* retrieving:

- **Rewriting** — turn a conversational question into a clean query. *"how's Tesla doing?"* → *"Tesla 2025 financial performance revenue profit risk factors"*.
- **Decomposition** — split a multi-part question into sub-queries, retrieve each, combine. *"Compare Tesla and Ford's EV risks"* → two queries. (This seeds multi-document RAG and feels agentic — it carries into Module 3.)

- **HyDE** (Hypothetical Document Embeddings) — ask the LLM to write a *fake answer*, embed *that*, and retrieve with it. The hypothetical answer reads like a real filing passage, so it lands closer in vector space than the terse question would.

The context-dependent case is constant in a chat copilot: a follow-up like *"what about their supply chain?"* is meaningless to a retriever without the prior turn — you rewrite it using history into a standalone query: *"Tesla supply chain risk factors 2025"*.

How these stack



The full path. Build naive RAG first, then add re-ranking (biggest win), then hybrid, then rewriting.

The unavoidable question: how do you *know* it improved?

Every pattern here is a *claimed* improvement — but "I added re-ranking and it feels better" is not engineering. You measure it: build a small set of question→correct-chunk pairs (a **golden dataset**) and check whether each change retrieves the right chunks more often (recall@k, precision@k) and whether answers are more faithful to sources. That measurement discipline — golden datasets, faithfulness scoring, LLM-as-a-judge — is **Module 5**. For now, plant the flag: *don't trust a RAG improvement you haven't measured.*

MENTAL MODEL TO CARRY FORWARD

- Naive RAG fails three ways: **missed exact terms, blunt cut, messy queries.**
- **Re-ranking** = retrieve wide (bi-encoder) then narrow (cross-encoder). Highest-ROI single upgrade.
- **Hybrid search** = dense + sparse (BM25), fused with RRF. Catches meaning *and* exact strings.
- **Query rewriting** (rewrite / decompose / HyDE) fixes bad queries before retrieval.
- Stack order: **rewrite → hybrid → re-rank → generate.**
- **Measure every change** — that's Module 5, and it's not optional.

Papers to read alongside this guide:

Lewis et al. 2020, *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* — arXiv:2005.11401
Karpukhin et al. 2020, *Dense Passage Retrieval for Open-Domain Question Answering* — arXiv:2004.04906
Liu et al. 2023, *Lost in the Middle: How Language Models Use Long Contexts* — arXiv:2307.03172